

Soccer Bot Project

By: Muhammad Haseeb

January 12, 2024

Contents

1	Introduction	3
2	Components	3
2.1	ESP32-WROOM-32	3
2.2	L298N Motor Driver	4
2.3	Geared Motors and Tires	4
2.4	Power Supply	4
3	Assembly of Components	5
4	Working Principle	5
4.1	Circuit Operation	5
4.2	Working Mechanism of Soccer Bot	6
5	Code Implementation	6
6	Flowcharts	10
6.1	Code Execution Flow	10
7	Conclusion	11

1 Introduction

This project details the design and implementation of a **Soccer Bot** using an **ESP32 microcontroller** and an **L298N motor driver module**. The bot is engineered for wireless motion control via Bluetooth communication, enabling it to move forward, backward, left, and right. It is powered by 18650 lithium-ion cells and utilizes geared DC motors with rubber tires for robust mobility.

2 Components

2.1 ESP32-WROOM-32

- Dual-core 32-bit LX6 processors with a clock speed of up to 240 MHz.
- 520 KB of SRAM and 4 MB of Flash memory.
- 32 configurable GPIO pins.
- Integrated Wi-Fi 802.11 b/g/n/e/i and Bluetooth v4.2.
- Power options: 3.3V or 5V via a built-in voltage regulator.

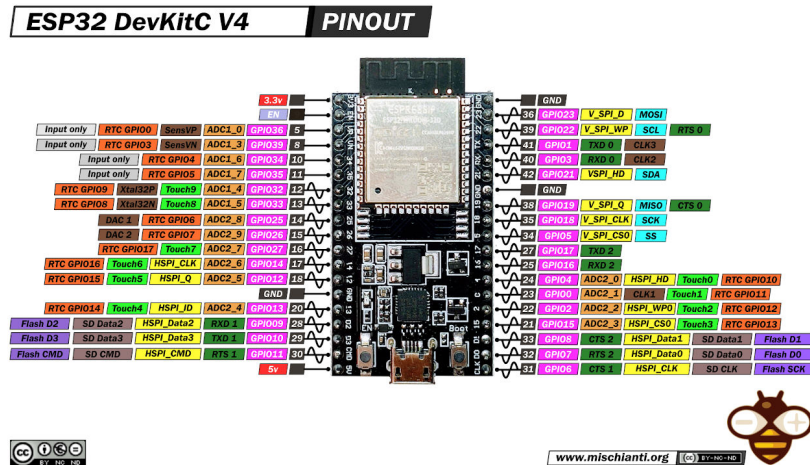


Figure 1: The ESP32-WROOM-32 module, the core of the bot's control system.

2.2 L298N Motor Driver

- A dual H-bridge motor driver capable of controlling two DC motors independently.
- Operates at 5V, with a built-in voltage regulator.
- Six control input pins for motor direction and speed control.
- Supports **Pulse Width Modulation (PWM)** for precise speed control.

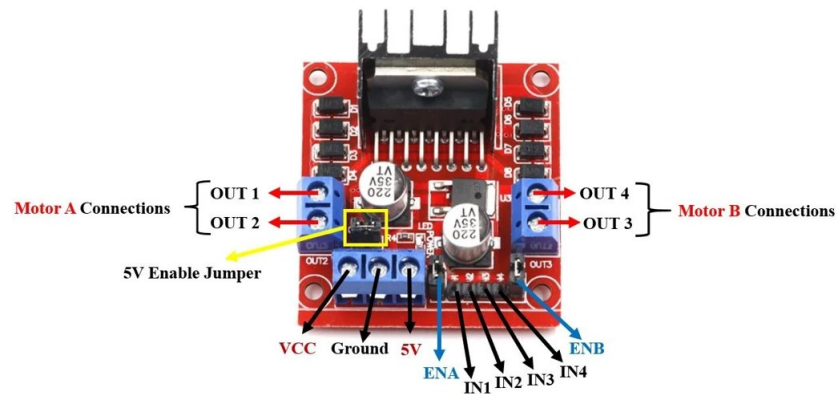


Figure 2: The L298N motor driver module used to power and control the geared motors.

2.3 Geared Motors and Tires

- Four geared DC motors (30–300 RPM range) for torque and controlled movement.
- Four rubber tires (65mm diameter) to provide grip and stability.

2.4 Power Supply

- A pack of four 18650 lithium-ion cells.
- This configuration provides sufficient voltage and current to power both the ESP32 and the high-torque motors.



Figure 3: The geared DC motors with rubber tires, providing locomotion for the bot.

3 Assembly of Components

- The ESP32 and the motor driver are powered by the 18650 battery pack.
 - The output pins of the ESP32 (pins 19, 21, 22, 23) are connected to the input pins of the L298N Motor Driver.
 - The two motors on each side of the bot are connected in parallel to the motor driver's outputs.
 - The L298N motor driver interprets the signals from the ESP32 to control the direction and speed of the motors, enabling the bot's movement.
-

4 Working Principle

4.1 Circuit Operation

- The battery pack supplies power to the ESP32 microcontroller and the L298N Motor Driver.
- The ESP32 generates control signals on its designated GPIO pins.



Figure 4: The 18650 lithium-ion battery pack used as the primary power source.

- The L298N Motor Driver receives and interprets these signals to adjust the polarity and voltage supplied to the motors.
- The motors on each side of the bot work in parallel to achieve movement, with differential drive steering enabling turns.

4.2 Working Mechanism of Soccer Bot

The Soccer Bot is controlled wirelessly via a Bluetooth connection. A command sent from a smartphone or computer is received by the ESP32. Based on this command (e.g., 'F' for forward), the ESP32 sends the appropriate digital signals to the L298N driver, which in turn drives the motors to achieve the desired movement.

5 Code Implementation

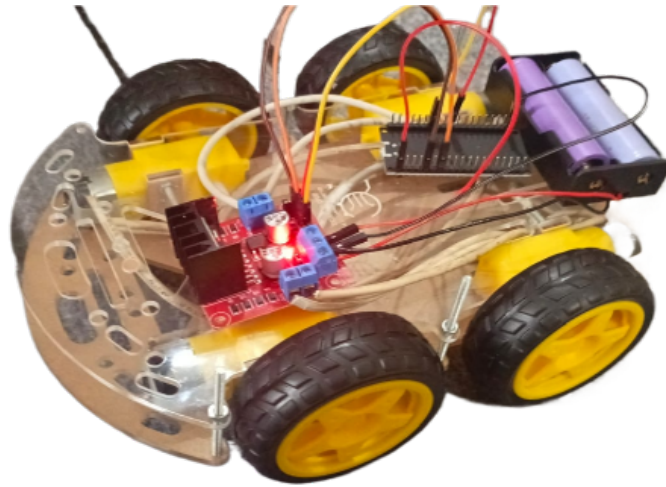


Figure 5: Block diagram illustrating the assembly and connections of the Soccer Bot's main components.

```

1  #include <SPI.h>
2  #include <Adafruit_GFX.h>
3  #include <Adafruit_ST7735.h>
4
5  #define TRIG_PIN 5
6  #define ECHO_PIN 22
7
8  #define TFT_CS 15
9  #define TFT_RST 4
10 #define TFT_DC 2
11
12 Adafruit_ST7735 tft = Adafruit_ST7735(TFT_CS, TFT_DC, TFT_RST
    );
13
14 float previousDistance = 0;
15 unsigned long previousTime = 0;
16
17 // Car X position
18 int carX = 10;
19
20 void setup() {
21     Serial.begin(115200);
22
23     pinMode(TRIG_PIN, OUTPUT);
24     pinMode(ECHO_PIN, INPUT);
25
26     tft.initR(INITR_BLACKTAB);

```

```

27 tft.setRotation(1);
28 tft.fillScreen(ST77XX_BLACK);
29 tft.setTextSize(2); // Bigger font
30 tft.setTextColor(ST77XX_WHITE);
31 tft.setCursor(10, 20);
32 tft.println("Ultrasonic Ready");
33 delay(1000);
34 tft.fillScreen(ST77XX_BLACK);
35 }
36
37 float getDistance() {
38     digitalWrite(TRIG_PIN, LOW);
39     delayMicroseconds(2);
40     digitalWrite(TRIG_PIN, HIGH);
41     delayMicroseconds(10);
42     digitalWrite(TRIG_PIN, LOW);
43
44     long duration = pulseIn(ECHO_PIN, HIGH, 30000);
45     if (duration == 0) return -1;
46     return duration * 0.0343 / 2.0;
47 }
48
49 void drawCar(int x, int y) {
50     // Simple car: rectangle + wheels
51     tft.fillRect(x, y, 30, 10, ST77XX_RED); // Body
52     tft.fillCircle(x + 5, y + 10, 3, ST77XX_WHITE); // Wheel 1
53     tft.fillCircle(x + 25, y + 10, 3, ST77XX_WHITE); // Wheel 2
54 }
55
56 void loop() {
57     float currentDistance = getDistance();
58     unsigned long currentTime = millis();
59     float timeDiff = (currentTime - previousTime) / 1000.0;
60
61     if (currentDistance >= 0 && timeDiff > 0.1) {
62         float speed = (previousDistance - currentDistance) /
        timeDiff;
63
64         Serial.print("Distance: ");
65         Serial.print(currentDistance);
66         Serial.print(" cm\tSpeed: ");
67         Serial.println(speed);
68
69         // Calculate car movement (scaled for visibility)
70         int movement = constrain((int)(speed * 2), -5, 5); //
        Speed scale
71
72         carX += movement;
73         if (carX < 0) carX = 0;

```



```

74     if (carX > 100) carX = 0; // Reset to left when out of
    screen
75
76     tft.fillScreen(ST77XX_BLACK);
77
78     // Distance
79     tft.setTextSize(2);
80     tft.setCursor(10, 5);
81     tft.setTextColor(ST77XX_CYAN);
82     tft.print("D:");
83     tft.print(currentDistance, 1);
84     tft.print("cm");
85
86     // Speed
87     tft.setCursor(10, 30);
88     tft.setTextColor(ST77XX_YELLOW);
89     tft.print("S:");
90     tft.print(speed, 2);
91     tft.print("cm/s");
92
93     // Draw car
94     drawCar(carX, 60);
95
96     previousDistance = currentDistance;
97     previousTime = currentTime;
98 }
99
100 delay(150);
101 }

```

6 Flowcharts

6.1 Code Execution Flow

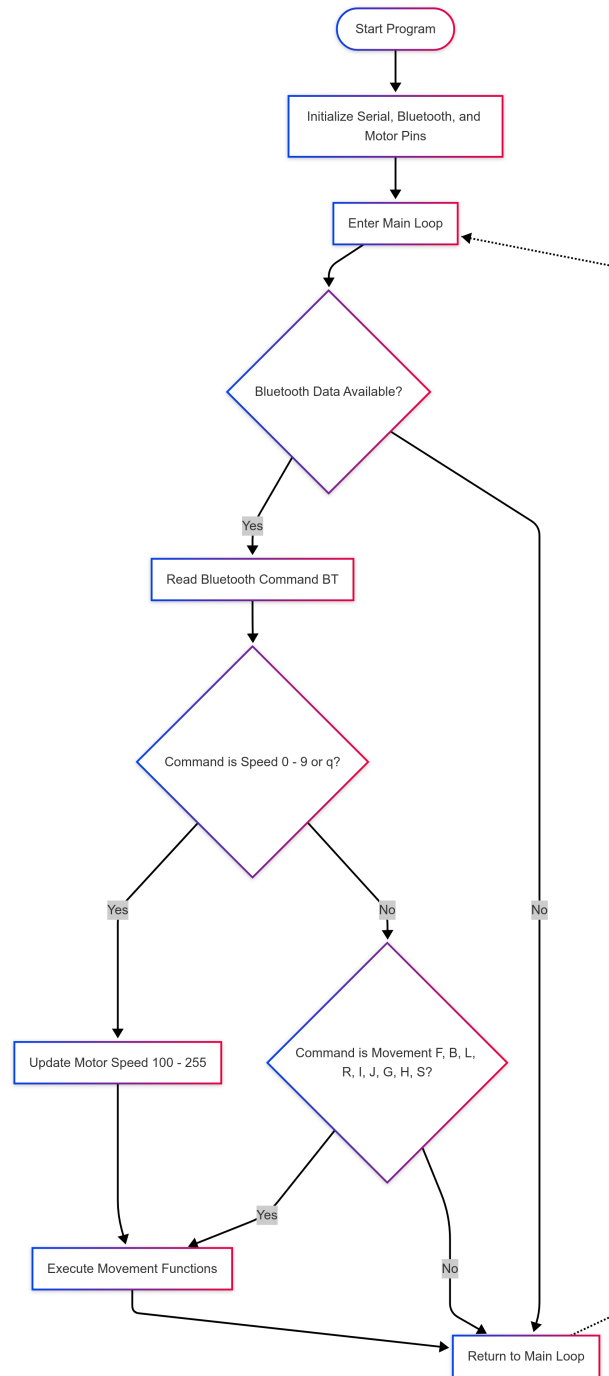


Figure 6: Code Execution Flow of the Soccer Bot's control loop.

7 Conclusion

The ****Soccer Bot**** project successfully demonstrated the wireless control of a robotic vehicle using an ESP32 and Bluetooth communication. The system provides a robust and scalable foundation for a wide range of robotics applications, including autonomous navigation, telepresence, and competitive robotic sports like soccer. This project serves as a practical example of integrating microcontroller technology with wireless communication for real-world applications.